Python Engineering - EX24

WEEK 04 | CLASS 16 - Testing & Project Structure

Objetivo: cerrar el bootcamp construyendo una pequeña estructura de proyecto verificable. Vamos a separar código de aplicación y tests, comprobar resultados automáticamente y distinguir entre un test que falla y un error inesperado del propio programa.

> La misión no es instalar un framework de testing. Primero construiremos el mecanismo básico a mano para entender qué está ocurriendo: **Arrange -> Act -> Assert**.

Ruta exacta

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_16_testing_project_structure/
        |-- notes/
        `-- exercises/
            `-- ex24_testing_project_structure/
                |-- vertical_state.py
                |-- file_io.py
                |-- main.py
                |-- run_tests.py
                |-- tests/
                |   |-- __init__.py
                |   |-- test_vertical_state.py
                |   `-- test_file_io.py
                `-- data/
```

Qué reutilizamos

Reutiliza las versiones limpias de `VerticalState` y de las funciones de File I/O de EX22/EX23. No copies los bloques de demostración o prints de los ejercicios anteriores: aquí queremos código de aplicación reutilizable.

Archivo	Responsabilidad
vertical_state.py	Modelo físico y estado vertical
file_io.py	Guardar/cargar trayectorias CSV
main.py	Ejemplo de uso de la aplicación
tests/*.py	Comprobaciones automáticas
run_tests.py	Ejecutar y resumir todos los tests

Valores de referencia

```
initial state:
h = 100.0 m
v = 20.0 m/s
t = 0.0 s
g = 9.81 m/s^2
dt = 0.1 s

after one Euler step:
h = 102.0
v = 19.019
t = 0.1
```

BLOCK A - Crear tests como funciones normales

En `tests/test_vertical_state.py`:

```
import numpy as np

from vertical_state import VerticalState


def test_initial_state():
    # ARRANGE
    state = VerticalState(h=100.0, v=20.0)

    # ACT
    result = state.state_vector()

    # ASSERT
    expected = np.array([100.0, 20.0, 0.0])
    assert np.allclose(result, expected), (
        f'Expected {expected}, got {result}'
    )
```

Un test es simplemente código que ejecuta una parte del programa y compara el resultado real con un resultado esperado.

BLOCK B - Test del método Euler

```
def test_euler_step():
    state = VerticalState(h=100.0, v=20.0)
    dt = 0.1

    state.euler_step(dt)

    result = state.state_vector()
    expected = np.array([102.0, 19.019, 0.1])

    assert np.allclose(result, expected), (
        f'Expected {expected}, got {result}'
    )
```

Aquí estamos comprobando un comportamiento físico/numerico conocido. Si en el futuro modificas `euler_step()` y rompes el resultado, el test te avisará.

BLOCK C - Por qué np.allclose() y no ==

Para números de coma flotante evitamos depender de igualdad exacta.

```
assert np.allclose(result, expected)
# better for floating-point numerical results
```

No significa que cualquier error sea aceptable: significa que comprobamos igualdad dentro de tolerancias numéricas pequeñas.

BLOCK D - Test de ida y vuelta de File I/O

En `tests/test_file_io.py`:

```
from pathlib import Path
import numpy as np

from file_io import save_trajectory_csv, load_trajectory_csv


def test_csv_round_trip():
    base_dir = Path(__file__).resolve().parent.parent
    data_dir = base_dir / 'data'
    data_dir.mkdir(exist_ok=True)

    test_file = data_dir / 'test_round_trip.csv'

    original = np.array([
        [100.0, 20.0, 0.0],
        [102.0, 19.019, 0.1]
    ])

    save_trajectory_csv(test_file, original)
    loaded = load_trajectory_csv(test_file)

    assert original.shape == loaded.shape
    assert np.allclose(original, loaded)

    test_file.unlink(missing_ok=True)
```

El test crea su propio archivo, lo utiliza y lo elimina al final. Esto reduce la dependencia de archivos manuales existentes.

BLOCK E - Test de una excepción esperada

```
def test_missing_file_raises():
    base_dir = Path(__file__).resolve().parent.parent
    missing_file = base_dir / 'data' / 'does_not_exist.csv'

    try:
        load_trajectory_csv(missing_file)

    except FileNotFoundError:
        pass

    else:
        assert False, (
            'Expected FileNotFoundError, but no exception was raised'
        )
```

Un comportamiento correcto no siempre significa 'no hubo excepción'. A veces el comportamiento correcto es precisamente que una entrada inválida genere la excepción apropiada.

BLOCK F - __init__.py

Crea un archivo vacío: `tests/__init__.py`. Para este ejercicio solo necesitamos saber que permite tratar `tests` como un paquete importable desde `run_tests.py`.

```
# tests/__init__.py
# empty file
```

BLOCK G - Crear nuestro pequeño test runner

En `run_tests.py`:

```
from tests.test_vertical_state import (
    test_initial_state,
    test_euler_step,
)
from tests.test_file_io import (
    test_csv_round_trip,
    test_missing_file_raises,
)


TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
]


def main():
    passed = 0
    failed = 0
    errors = 0

    for test in TESTS:
        try:
            test()

        except AssertionError as error:
            failed += 1
            print(f'FAIL  {test.__name__}')
            print(f'      {error}')

        except Exception as error:
            errors += 1
            print(f'ERROR {test.__name__}')
            print(f'      {type(error).__name__}: {error}')

        else:
            passed += 1
            print(f'PASS  {test.__name__}')

    print('\nSUMMARY')
    print(f'Passed: {passed}')
    print(f'Failed: {failed}')
    print(f'Errors: {errors}')


if __name__ == '__main__':
    main()
```

Aquí `except Exception` aparece deliberadamente en el **límite del test runner**: queremos evitar que un error inesperado detenga el resto de tests y queremos etiquetarlo como ERROR. No estamos ocultándolo.

BLOCK H - Ejecutar desde la raíz del ejercicio

```
cd .../ex24_testing_project_structure
python run_tests.py
```

Resultado esperado:

```
PASS  test_initial_state
PASS  test_euler_step
PASS  test_csv_round_trip
PASS  test_missing_file_raises

SUMMARY
Passed: 4
Failed: 0
Errors: 0
```

BLOCK I - Romper algo a propósito

Modifica temporalmente un resultado esperado para observar un `FAIL`. Por ejemplo:

```
expected = np.array([999.0, 19.019, 0.1])
```

Ejecuta los tests, observa el fallo y después restaura el valor correcto. Esta prueba deliberada demuestra que el test realmente detecta un cambio incorrecto.

BLOCK J - FAIL no es lo mismo que ERROR

Resultado	Qué significa
PASS	El comportamiento observado coincide con lo esperado
FAIL	El test se ejecutó, pero una assertion no se cumplió
ERROR	El test no pudo completarse por una excepción inesperada

FINAL VALIDATION - responder en comentarios

```
- What is a test?
- What is the difference between production/application code and test code?
- Explain Arrange, Act and Assert.
- What does assert do?
- What happens when an assertion is False?
- Why do we use np.allclose for floating-point arrays?
- Why should each test check one clear behavior?
- What is a regression?
- Why should tests be repeatable?
- What is the difference between PASS, FAIL and ERROR?
- Why do we intentionally test FileNotFoundError?
- Why is except Exception acceptable in our test runner but not a good default everywhere?
- What is tests/__init__.py doing in this exercise?
- Why do tests live in a separate folder?
- Why should main.py not contain the test logic?
- What advantage does this structure give us before uploading to GitHub?
```

Criterio de cierre

EX24 estará completado cuando los cuatro tests pasen, puedas provocar voluntariamente al menos un FAIL, entiendas la diferencia entre FAIL y ERROR y puedas explicar por qué la estructura separa aplicación, datos y tests.